ECC Circuit

개요

ECC Circuit은 EVM의 타원곡선 암호(Elliptic Curve Cryptography) 프리컴파일 호출을 검증하는 회로입니다. 이 회로는 BN254(alt_bn128) 곡선에서 세 가지 주요 연산을 지원합니다:

1. **EcAdd (점 덧셈)**: P + Q = R
2. **EcMul (스칼라 곱셈)**: s·P = R
3. **EcPairing (페어링)**: e(G1, G2) 검증

이 회로는 `halo2-ecc` 라이브러리를 기반으로 하며, BN254 곡선의 G1, G2 그룹 연산과 페어링 검증을 효율적으로 처리합니다.

주요 특징:

- BN254 곡선 지원 (Ethereum 표준)
- CRT(Chinese Remainder Theorem) 정수 표현
- 2단계 증명 (Phase 1: 연산 검증, Phase 2: RLC 계산)
- 엣지 케이스 처리 (무한원점, 유효하지 않은 입력)
- 고정된 최대 연산 수 (미리 할당된 셀)

파일 위치:

- `zkevm-circuits/src/ecc_circuit.rs` - 메인 회로 구조
- `zkevm-circuits/src/ecc_circuit/util.rs` - 데이터 구조 정의
- `zkevm-circuits/src/ecc_circuit/test.rs` - 테스트 코드

BN254 곡선 개요

BN254(Barreto-Naehrig) 곡선은 페어링 기반 암호에 최적화된 타원곡선입니다.

곡선 정의

G1 (Fq 위의 곡선):

```
y² = x³ + 3
```

G2 (Fq^2 위의 곡선):

```
y² = x³ + 3/(9+u)
```

여기서 Fq^2는 $Fq[u]/(u^2+1)$로 정의되는 확장 필드입니다.

필드 크기

- **Fq (기본 필드)**: 254비트 소수
- **Fr (스칼라 필드)**: 254비트 소수
- **Fq^2 (확장 필드)**: 2개의 Fq 원소로 구성
- **Fq^{12} (타겟 필드)**: 12개의 Fq 원소로 구성

페어링 함수

BN254는 최적 Ate 페어링을 사용합니다:

```
e: G1 × G2 → GT (Fq¹²의 부분군)
```

페어링 속성:

- 쌍선형성: e(aP, bQ) = e(P, Q)^(ab)

- 비퇴화성: e(P, Q) ≠ 1 (P, Q ≠ 0인 경우)

회로 구조

EccCircuitConfig

`EccCircuitConfig` 는 ECC Circuit의 설정을 정의합니다:

```
pub struct EccCircuitConfig<F: Field> {
fp_config: FpConfig<F, Fq>, // Fq 필드 연산 설정
ecc_table: EccTable, // ECC 테이블 (I/O)
num_limbs: usize, // CRT limb 개수 (3)
limb_bits: usize, // limb당 비트 수 (88)
_marker: PhantomData<F>,
}
```

FpConfig 설정

`FpConfig` 는 `halo2-lib` 의 필드 연산 칩입니다:

파라미터:

- **num_limbs**: 3 (88비트 × 3 = 264비트 > 254비트)
- **limb_bits**: 88
- **num_advice**: [35] (onephase) 또는 [35, 1] (twophase)
- **num_lookup_advice**: [17]
- **lookup_bits**: 13 (8K 룩업 테이블)
- **k**: LOG_TOTAL_NUM_ROWS (20)

EccCircuit 구조

```
pub struct EccCircuit<F: Field, const XI_0: i64> {
max_add_ops: usize, // 최대 EcAdd 연산 수
max_mul_ops: usize, // 최대 EcMul 연산 수
max_pairing_ops: usize, // 최대 EcPairing 연산 수
add_ops: Vec<EcAddOp>, // 실제 EcAdd 연산들
mul_ops: Vec<EcMulOp>, // 실제 EcMul 연산들
pairing_ops: Vec<EcPairingOp>, // 실제 EcPairing 연산들
_marker: PhantomData<F>,
}
```

XI_0 상수:

- Fq^{12} 확장 필드의 비가역 원소 정의
- BN254의 경우 XI_0 = 9

데이터 구조

CRT 정수 표현

ECC Circuit은 큰 정수를 CRT(Chinese Remainder Theorem) 형식으로 표현합니다:

```
pub struct CRTInteger<F: Field> {
truncation: OverflowInteger<F>, // limb들의 배열
native: AssignedValue<F>, // 네이티브 필드 값
```

```
value: Value<BigInt>, // 실제 BigInt 값
}
```

Limb 분해 예시 (254비트 값):

```
value = limb[0] + limb[1] * 2^88 + limb[2] * 2^176
```

각 limb는 88비트로 제한되며, 3개의 limb로 최대 264비트를 표현할 수 있습니다.

G1 점 구조

G1Decomposed (Phase 1):

```
pub struct G1Decomposed<F: Field> {
ec_point: EcPoint<F, CRTInteger<F>>, // (x, y) CRT 표현
x_cells: Vec<QuantumCell<F>>, // x 좌표의 LE 바이트들
y_cells: Vec<QuantumCell<F>>, // y 좌표의 LE 바이트들
}
```

G1Assigned (Phase 2):

```
pub struct G1Assigned<F: Field> {
x_rlc: AssignedValue<F>, // x 좌표의 RLC
y_rlc: AssignedValue<F>, // y 좌표의 RLC
}
```

G2 점 구조

G2Decomposed:

```
pub struct G2Decomposed<F: Field> {
ec_point: EcPoint<F, FieldExtPoint<CRTInteger<F>>>,
x_c0_cells: Vec<QuantumCell<F>>, // x의 계수 0 (LE 바이트)
x_c1_cells: Vec<QuantumCell<F>>, // x의 계수 1 (LE 바이트)
y_c0_cells: Vec<QuantumCell<F>>, // y의 계수 0 (LE 바이트)
y_c1_cells: Vec<QuantumCell<F>>, // y의 계수 1 (LE 바이트)
}
```

G2 점은 Fq^2 위에 정의되므로 각 좌표가 2개의 Fq 원소로 구성됩니다.

스칼라 구조

```
pub struct ScalarAssigned<F: Field> {
scalar: CRTInteger<F>, // Fr 필드의 스칼라
}
```

스칼라는 Fr 필드에 속하므로 네이티브 필드에 맞으며, Phase 1에서 완전히 할당됩니다.

EcAdd 연산 (점 덧셈)

개요

EcAdd는 G1 곡선 위의 두 점 P와 Q를 더하여 R을 계산합니다:

```
P + Q = R
```

입력 검증

각 점 P, Q에 대해 다음을 검증합니다:

1. **필드 범위 검증**: x, y < Fq의 모듈러스
2. **곡선 방정식 검증**: $y^2 = x^3 + 3$
3. **무한원점 처리**: (0, 0)을 무한원점으로 간주

```
fn  precheck_fq(
&self,
ctx: &mut Context<F>,
ecc_chip: &EccChip<F, FpConfig<F, Fq>>,
word_value: U256,
powers_of_256: &[QuantumCell<F>],
) -> (
CRTInteger<F>, // CRT 표현
Vec<QuantumCell<F>>, // LE 바이트들
AssignedValue<F>, // value < Fq::MODULUS
AssignedValue<F>, // value == 0
)
```

곡선 검증

```
fn  is_on_curveg1_or_infinity(
```

```
&self,
ctx: &mut Context<F>,
ecc_chip: &EccChip<F, FpConfig<F, Fq>>,
x: &CRTInteger<F>,
x_is_zero: AssignedValue<F>,
y: &CRTInteger<F>,
y_is_zero: AssignedValue<F>,
) -> AssignedValue<F>
```

검증 로직:

```
lhs = y²
rhs = x³ + 3
is_on_curve = (lhs - rhs == 0)
is_infinity = (x == 0) AND (y == 0)
result = is_on_curve OR is_infinity
```

덧셈 검증 전략

단순히 P + Q = R을 검증하는 대신, 다음 방정식을 사용합니다:

```
r + P + Q - R = r
```

여기서 r은 곡선 위의 무작위 점입니다.

이유:

- P = (0, 0) 또는 Q = (0, 0) 처리
- P = -Q인 경우 (P + Q = 무한원점) 처리
- 고정된 제약 조건으로 모든 엣지 케이스 커버

구현 세부사항

```
fn  decompose_ec_add_op(
&self,
ctx: &mut Context<F>,
ecc_chip: &EccChip<F, FpConfig<F, Fq>>,
// ...
op: &EcAddOp,
) -> EcAddDecomposed<F>
```

처리 단계:

1. **입력 검증**:

```
let (px, px_cells, px_valid, px_is_zero) =
self.precheck_fq(ctx, ecc_chip, op.p.0, powers_of_256);
let (py, py_cells, py_valid, py_is_zero) =
self.precheck_fq(ctx, ecc_chip, op.p.1, powers_of_256);
let  p_is_on_curve_or_infinity =
self.is_on_curveg1_or_infinity(ctx, ecc_chip, &px, px_is_zero, &py, py_is_zero);
```

2. **유효성 플래그 계산**:

```
let inputs_valid = ecc_chip.field_chip().range().gate().and_many(
ctx,
vec![
px_valid, py_valid, p_is_on_curve_or_infinity,
qx_valid, qy_valid, q_is_on_curve_or_infinity,
],
);
```

3. **무작위 점 생성**:

```
let rand_point = ecc_chip.load_random_point::<G1Affine>(ctx);
```

4. **조건부 덧셈**:

```
// sum1 = if P == (0, 0) then r else r + P
let sum1 = ecc_chip.add_unequal(ctx, &rand_point, &point_p, true);
let sum1 = ecc_chip.select(ctx, &rand_point, &sum1, &point_p_is_zero);
// sum2 = if Q == (0, 0) then sum1 else sum1 + Q
let sum2 = ecc_chip.add_unequal(ctx, &sum1, &point_q, true);
let sum2 = ecc_chip.select(ctx, &sum1, &sum2, &point_q_is_zero);
// sum3 = if R == (0, 0) then sum2 else sum2 - R
let sum3 = ecc_chip.sub_unequal(ctx, &sum2, &point_r.ec_point, true);
let sum3 = ecc_chip.select(ctx, &sum2, &sum3, &point_r_is_zero);
```

5. **최종 검증**:

```
ecc_chip.assert_equal(ctx, &rand_point, &sum3);
```

셀 사용량

- **EC_ADD_CELLS**: 약 6,900 셀 (실제: 6,851)
- 주로 필드 연산과 범위 검증에 사용

EcMul 연산 (스칼라 곱셈)

개요

EcMul은 G1 곡선 위의 점 P에 스칼라 s를 곱하여 R을 계산합니다:

```
s·P = R
```

입력 검증

1. **점 P 검증**: EcAdd와 동일한 방식
2. **스칼라 s 검증**: Fr 필드 원소 (자동으로 범위 내)

스칼라 곱셈 알고리즘

`halo2-ecc` 의 `scalar_mult` 메서드를 사용합니다:

```
let  point_r_got = ecc_chip.scalar_mult::<G1Affine>(
ctx,
&point_p,
```

```
&scalar_s.scalar.limbs().to_vec(),
fr_chip.limb_bits, // 88
4, // window_bits
);
```

알고리즘:

- Window NAF (Non-Adjacent Form) 방식
- Window 크기: 4비트
- 사전 계산된 테이블 사용
- 더블-앤-애드(double-and-add) 최적화

무한원점 처리

입력이 유효하지 않은 경우:

```
let dummy_g1 = ecc_chip.load_random_point::<G1Affine>(ctx);
let point_p = ecc_chip.select(ctx, &point_p, &dummy_g1, &is_valid);

// 결과를 무한원점으로 설정
let infinity = EcPoint::construct(
ecc_chip.field_chip().load_constant(ctx, fe_to_biguint(&Fq::zero())),
ecc_chip.field_chip().load_constant(ctx, fe_to_biguint(&Fq::zero())),
);
let point_r_got = ecc_chip.select(ctx, &point_r_got, &infinity, &is_valid);
```

구현 세부사항

```
fn decompose_ec_mul_op(
&self,
ctx: &mut Context<F>,
ecc_chip: &EccChip<F, FpConfig<F, Fq>>,
fr_chip: &FpConfig<F, Fr>,
// ...
op: &EcMulOp,
) -> EcMulDecomposed<F>
```

처리 단계:

1. **점 P 검증** (EcAdd와 동일)
2. **스칼라 s 처리**:

```
let scalar_s = self.handle_fr(ctx, fr_chip, op.s);
```

3. **스칼라 곱셈 수행**
4. **결과 검증**:

```
ecc_chip.assert_equal(ctx, &point_r.ec_point, &point_r_got);
```

셀 사용량

- **EC_MUL_CELLS**: 약 405,500 셀 (실제: 405,476)

- 스칼라 곱셈이 가장 비용이 큰 연산

EcPairing 연산 (페어링 검증)

개요

EcPairing은 여러 (G1, G2) 쌍에 대한 페어링 곱이 1인지 검증합니다:

$$e(G1_1, G2_1) \cdot e(G1_2, G2_2) \cdot \ldots \cdot e(G1_n, G2_n) = 1$$

이는 다음과 동치입니다:

$$e(G1_1, G2_1) \cdot e(G1_2, G2_2) \cdot \ldots \cdot e(G1_{n-1}, G2_{n-1}) = e(-G1_n, G2_n)$$

입력 형식

EVM의 ecPairing 프리컴파일은 다음 형식의 입력을 받습니다:

$$[G1_1.x, G1_1.y, G2_1.x, G2_1.y, G1_2.x, G1_2.y, G2_2.x, G2_2.y, \ldots]$$

각 필드:

- G1.x, G1.y: 각 32바이트 (Fq)
- G2.x, G2.y: 각 64바이트 (Fq^2, 2개의 32바이트 계수)

총 192바이트 × N쌍 (N ≤ N_PAIRING_PER_OP)

입력 검증

각 (G1, G2) 쌍에 대해:

1. **G1 점 검증**:

- x, y < Fq 모듈러스
- $y^2 = x^3 + 3$
- 또는 (0, 0) = 무한원점

2. **G2 점 검증**:

- x_0, x_1, y_0, y_1 < Fq 모듈러스
- $y^2 = x^3 + 3/(9+u)$ (Fq^2 위에서)
- 또는 (0, 0, 0, 0) = 무한원점

3. **쌍 유효성**:

```
let  is_pair_valid = ecc_chip.field_chip().range().gate().and(
ctx,
QuantumCell::Existing(is_g1_valid),
QuantumCell::Existing(is_g2_valid),
);
```

4. **제로 쌍 검출**:

```
// G1 = 무한원점 AND G2 유효
// 또는 G2 = 무한원점 AND G1 유효
let  is_zero_pair = {
let  is_zero_pair = ecc_chip.field_chip().range().gate().or(
ctx,
QuantumCell::Existing(is_g1_identity),
QuantumCell::Existing(is_g2_identity),
```

```
);
ecc_chip.field_chip().range().gate().and(
ctx,
QuantumCell::Existing(is_zero_pair),
QuantumCell::Existing(is_pair_valid),
)
};
```

페어링 계산

halo2-ecc 의 `PairingChip` 을 사용합니다:

```
let pairing_chip = PairingChip::construct(config.fp_config.clone());
let fp12_chip = Fp12Chip::<F, FpConfig<F, Fq>, Fq12, XI_0>::construct(
config.fp_config.clone()
);
```

계산 단계:

1. **Multi-Miller Loop**:

```
let gt = pairing_chip.multi_miller_loop(ctx, pairs);
```

- 모든 쌍에 대한 Miller 루프를 동시에 계산
- 결과는 Fq^{12} 원소

2. **Final Exponentiation**:

```
let gt = pairing_chip.final_exp(ctx, &gt);
```

- Miller 루프 결과를 최종 지수화
- 결과가 GT 부분군에 속하도록 보장

3. **결과 검증**:

```
let one = fp12_chip.load_constant(ctx, Fq12::one());
let success = fp12_chip.is_equal(ctx, &gt, &one);
```

엣지 케이스 처리

모든 쌍이 제로 쌍인 경우

```
let all_pairs_zero = ecc_chip.field_chip().range().gate().and_many(
ctx,
decomposed_pairs
.iter()
.map(|(is_zero_pair, _, _, _)| QuantumCell::Existing(*is_zero_pair))
.collect_vec(),
);
```

이 경우 더미 페어링 체크를 사용:

```
let (dummy_pair_check_ok_g1s, dummy_pair_check_ok_g2s) =
EcPairingOp::dummy_pairing_check_ok()
```

```
.pairs
.iter()
.map(|pair| /* ... */)
.unzip();
```

유효하지 않은 쌍 처리

유효하지 않은 쌍은 더미 점으로 교체:

```
let swapped_g1 = ecc_chip.select(ctx, &g1.ec_point, &dummy_g1, is_pair_valid);
let swapped_g1 = ecc_chip.select(ctx, &identity_g1, &swapped_g1, is_zero_pair);
```

성공 플래그 계산

```
// success = is_valid AND pairing_check_ok
let success = ecc_chip.field_chip().range().gate().and(
ctx,
QuantumCell::Existing(is_valid),
QuantumCell::Existing(success),
);


// 모든 쌍이 제로인 경우 success = true
// success - all_pairs_zero는 boolean이어야 함
let success_minus_all_pairs_zero = ecc_chip
```

```
.field_chip()
.range()
.gate()
.load_witness(ctx, success.value - all_pairs_zero.value);
ecc_chip
.field_chip()
.range()
.gate()
.assert_bit(ctx, success_minus_all_pairs_zero);
```

셀 사용량

- **EC_PAIRING_CELLS**: 약 6,627,500 셀 (실제: 6,627,442)
- 페어링이 가장 비용이 큰 연산 (Miller 루프 + Final Exp)

2단계 증명 프로세스

ECC Circuit은 Halo2의 2단계 증명을 활용합니다.

Phase 1: 연산 검증

목적: 타원곡선 연산의 정확성 검증

작업:

1. 입력 점들의 필드 범위 검증
2. 곡선 방정식 만족 검증
3. 점 덧셈/스칼라 곱셈/페어링 계산

4. 결과 검증

5. 바이트 분해 (LE 형식)

출력:

- `EcAddDecomposed`, `EcMulDecomposed`, `EcPairingDecomposed`
- 각 좌표의 바이트 셀들 (`x_cells`, `y_cells` 등)

Phase 2: RLC 계산

목적: ECC 테이블에 기록할 RLC 값 계산

작업:

1. Keccak challenge 값 사용
2. 바이트 셀들의 RLC 계산
3. ECC 테이블에 복사

RLC 계산 예시 (EcAdd):

```
fn  assign_ec_add(
&self,
ctx: &mut Context<F>,
ec_add_decomposed: &EcAddDecomposed<F>,
ecc_chip: &EccChip<F, FpConfig<F, Fq>>,
keccak_powers: &[QuantumCell<F>],
) -> EcAddAssigned<F> {
EcAddAssigned {
is_valid: ec_add_decomposed.is_valid,
point_p: G1Assigned {
```

```
x_rlc: ecc_chip.field_chip().range().gate().inner_product(
ctx,
ec_add_decomposed.point_p.x_cells.clone(),
keccak_powers.iter().cloned(),
),
y_rlc: ecc_chip.field_chip().range().gate().inner_product(
ctx,
ec_add_decomposed.point_p.y_cells.clone(),
keccak_powers.iter().cloned(),
),
},
// point_q, point_r도 동일
}
}
```

Keccak Powers:

```
let  keccak_powers = std::iter::successors(
Some(Value::known(F::one())),
|coeff| Some(challenges.keccak_input() * coeff)
)
.take(N_PAIRING_PER_OP * N_BYTES_PER_PAIR)
.map(|x| QuantumCell::Witness(x))
```

```
.collect_vec();
```

Phase 구분 (Feature Flag)

```
#[cfg(not(feature = "onephase"))]
{
config.fp_config.finalize(&mut  ctx);
ctx.next_phase();
}
```

- **onephase**: 단일 단계 증명 (테스트용)
- **twophase** (기본): 2단계 증명 (프로덕션)

ECC 테이블 통합

테이블 스키마

ECC 테이블은 9개의 컬럼으로 구성됩니다:

컬럼	타입	설명
op_type	Fixed	연산 타입 (EcAdd/EcMul/EcPairing)
is_valid	Advice	입력 유효성 플래그
arg1_rlc	Advice (Phase 2)	인자 1 RLC
arg2_rlc	Advice (Phase 2)	인자 2 RLC
arg3_rlc	Advice (Phase 2)	인자 3 RLC

| arg4_rlc | Advice (Phase 2) | 인자 4 RLC |

| input_rlc | Advice (Phase 2) | 전체 입력 RLC |

| output1_rlc | Advice (Phase 2) | 출력 1 RLC |

| output2_rlc | Advice (Phase 2) | 출력 2 RLC |

연산별 테이블 매핑

EcAdd (Bn128Add)

```
op_type = PrecompileCalls::Bn128Add

is_valid = inputs_valid

arg1_rlc = RLC(P.x)

arg2_rlc = RLC(P.y)

arg3_rlc = RLC(Q.x)

arg4_rlc = RLC(Q.y)

input_rlc = 0

output1_rlc = RLC(R.x)

output2_rlc = RLC(R.y)
```

EcMul (Bn128Mul)

```
op_type = PrecompileCalls::Bn128Mul

is_valid = inputs_valid

arg1_rlc = RLC(P.x)

arg2_rlc = RLC(P.y)
```

```
arg3_rlc = scalar_s (네이티브 필드 값)

arg4_rlc = 0

input_rlc = 0

output1_rlc = RLC(R.x)

output2_rlc = RLC(R.y)
```

EcPairing (Bn128Pairing)

```
op_type = PrecompileCalls::Bn128Pairing

is_valid = inputs_valid

arg1_rlc = 0

arg2_rlc = 0

arg3_rlc = 0

arg4_rlc = 0

input_rlc = RLC(모든 입력 바이트)

output1_rlc = success (0 또는 1)

output2_rlc = 0
```

테이블 할당

```
layouter.assign_region(

|| "expose ecc table",

|mut  region| {

// EcAdd 연산들
```

```
for (idx, ec_add_assigned) in  assigned_ec_ops.ec_adds_assigned.iter().enumerate

region.assign_fixed(

|| "assign ecc_table op_type",

config.ecc_table.op_type,

idx,

|| Value::known(F::from(u64::from(PrecompileCalls::Bn128Add))),

)?;

ec_add_assigned.is_valid.copy_advice(

&mut  region,

config.ecc_table.is_valid,

idx,

);

// arg1_rlc ~ output2_rlc 복사

}

// EcMul, EcPairing도 동일한 방식

},

)?;
```

최적화 기법

1. 고정된 최대 연산 수

회로는 미리 정해진 최대 연산 수를 지원합니다:

```
pub struct EccCircuit<F: Field, const XI_0: i64> {
pub max_add_ops: usize,
pub max_mul_ops: usize,
pub max_pairing_ops: usize,
// ...
}
```

장점:

- 회로 크기 예측 가능
- 셀 할당 최적화
- 증명 시간 일정

처리:

```
// 실제 연산 + 더미 연산으로 최대 수 채우기
$ops.iter()
.filter(|op| !op.skip_by_ecc_circuit())
.chain(std::iter::repeat(&$op_type::default()))
.take($n_ops)
```

2. CRT 정수 표현

큰 정수를 여러 limb로 분해:

장점:

- 네이티브 필드 연산 활용

- 범위 검증 효율화
- Carry 처리 최적화

Limb 구성:

```
254비트 값 = limb[0] (88비트) + limb[1] (88비트) + limb[2] (88비트)
```

3. 조건부 선택 (Select)

`select` 가젯을 사용한 조건부 로직:

```
// if condition then a else b
let  result = ecc_chip.select(ctx, &a, &b, &condition);
```

장점:

- 분기 없이 고정된 제약 조건
- 모든 경로 동일한 비용
- 엣지 케이스 처리 간소화

4. 무작위 점 기법

EcAdd 검증에서 무작위 점 r 사용:

```
let  rand_point = ecc_chip.load_random_point::<G1Affine>(ctx);
```

장점:

- 단일 방정식으로 모든 케이스 처리
- P = 0, Q = 0, R = 0 등 엣지 케이스 자동 처리

- 제약 조건 수 감소

5. 더미 페어링 체크

모든 쌍이 제로인 경우 더미 페어링 사용:

```
let (dummy_pair_check_ok_g1s, dummy_pair_check_ok_g2s) =
EcPairingOp::dummy_pairing_check_ok()
```

장점:

- 페어링 계산 항상 성공
- 엣지 케이스 처리 간소화
- 제약 조건 일관성 유지

성능 특성

셀 사용량

| 연산 | 셀 수 | 비고 |

|------|-------|------|

| EcAdd | 6,900 | 필드 연산 + 범위 검증 |

| EcMul | 405,500 | 스칼라 곱셈 (window NAF) |

| EcPairing | 6,627,500 | Miller 루프 + Final Exp |

행 사용량

회로는 k=20 (2^{20} = 1,048,576행)을 사용합니다:

```
pub const LOG_TOTAL_NUM_ROWS: u32 = 20;
```

최대 연산 수 예시 (k=20 기준):

- EcAdd: 약 150개
- EcMul: 약 2-3개
- EcPairing: 약 1개 (6쌍)

컬럼 제한

```
pub const COLUMN_NUM_LIMIT: usize = 150;
```

FpConfig는 최대 150개의 advice 컬럼을 사용할 수 있습니다.

증명 시간 (예상)

- EcAdd 1개: 약 0.1초
- EcMul 1개: 약 5초
- EcPairing 1개 (6쌍): 약 60초

실제 시간은 하드웨어와 최적화 수준에 따라 다릅니다.

구현 예시

EcAdd 검증

```
// 회로 생성
let circuit = EccCircuit {
```

```
max_add_ops: 10,
max_mul_ops: 0,
max_pairing_ops: 0,
add_ops: vec![
EcAddOp {
p: (p_x, p_y),
q: (q_x, q_y),
r: Some(r_affine),
}
],
mul_ops: vec![],
pairing_ops: vec![],
_marker: PhantomData,
};

// 증명 생성
let prover = MockProver::run(k, &circuit, vec![])?;
prover.assert_satisfied();
```

EcMul 검증

```
let circuit = EccCircuit {
max_add_ops: 0,
```

```
max_mul_ops: 5,

max_pairing_ops: 0,

add_ops: vec![],

mul_ops: vec![

EcMulOp {

p: (p_x, p_y),

s: scalar,

r: Some(result),

}

],

pairing_ops: vec![],

_marker: PhantomData,

};
```

EcPairing 검증

```
let  circuit = EccCircuit {

max_add_ops: 0,

max_mul_ops: 0,

max_pairing_ops: 1,

add_ops: vec![],

mul_ops: vec![],

pairing_ops: vec![
```

```
EcPairingOp {
pairs: vec![
(g1_1, g2_1),
(g1_2, g2_2),
],
output: Fr::one(), // success
}
],
_marker: PhantomData,
};
```

프리컴파일 통합

EVM Circuit 연동

EVM Circuit에서 ECC 프리컴파일 호출 시:

1. **입력 수집**: CallData에서 점 좌표 추출
2. **ECC 테이블 룩업**:

```
// EcAdd 예시
lookup(
op_type = Bn128Add,
is_valid = 1,
arg1_rlc = RLC(P.x),
```

```
arg2_rlc = RLC(P.y),

arg3_rlc = RLC(Q.x),

arg4_rlc = RLC(Q.y),

) -> (output1_rlc, output2_rlc)
```

3. **결과 사용**: output RLC를 반환 데이터로 사용

Bus-Mapping 연동

Bus-Mapping에서 ECC 연산 수집:

```
// EcAddOp 생성

pub struct EcAddOp {

pub p: (U256, U256), // P.x, P.y

pub q: (U256, U256), // Q.x, Q.y

pub r: Option<G1Affine>, // 결과 (witness)

}


// Block에서 추출

impl Block {

pub fn get_ec_add_ops(&self) -> Vec<EcAddOp> {

// 트레이스에서 EcAdd 호출 수집

}

}
```

최대 연산 수 설정

```
pub struct CircuitsParams {
pub max_ec_ops: MaxEcOps,
// ...
}

pub struct MaxEcOps {
pub ec_add: usize,
pub ec_mul: usize,
pub ec_pairing: usize,
}
```

블록 파라미터에서 최대 연산 수를 설정하여 회로 크기를 제어합니다.

알려진 제약사항

1. 고정된 최대 연산 수

실제 사용하지 않는 연산도 더미 연산으로 채워야 합니다:

```
if self.add_ops.len() > self.max_add_ops {
error!("add ops = {} > max add ops = {}",
self.add_ops.len(), self.max_add_ops);
return Err(Error::Synthesis);
```

}

2. 큰 회로 크기

특히 EcPairing은 매우 많은 셀을 사용합니다:

- 1개 페어링 (6쌍): 약 6.6M 셀
- k=20에서 약 1개만 처리 가능

3. 증명 시간

EcPairing의 증명 시간이 매우 깁니다:

- Miller 루프: O(log r) 반복
- Final Exponentiation: Fq^{12} 연산

4. 메모리 사용량

큰 k 값과 많은 컬럼으로 인해 메모리 사용량이 높습니다:

- k=20, 150 컬럼: 약 150M 셀

관련 문서

- Sig Circuit - ECDSA 서명 검증 (secp256k1)
- EVM Circuit - 프리컴파일 호출
- ECC Table - ECC 테이블 스키마
- Lookup 메커니즘 - 테이블 룩업

참고 자료

- EIP-196 - EcAdd 프리컴파일
- EIP-197 - EcMul, EcPairing 프리컴파일
- BN254 Curve - BN254 곡선 설명
- halo2-ecc - ECC 가젯 라이브러리
- Optimal Ate Pairing - 페어링 알고리즘